

CVM++

Everything you need to understand, build, and run CVM++ — explained from zero

A self-contained guide — read top to bottom.

github.com/Dragon9-9/cvmpp · Satish Kumar Munda · MIT License

Who is this guide for?

You might be a reviewer, a student, or someone learning how programming languages work. You do **not** need to have built a compiler before. You **do** need to know how to open a terminal and run commands like `cd` and `make`.

This document answers:

1. **What** is CVM++ and **why** does it exist?
2. **How** do I download, build, and run it?
3. **What** does each folder and file in the project do?
4. **In what order** was the system built (so you can learn or extend it)?
5. **What** can I write in the `.cvm` language?
6. **What do all the technical words mean?**

There is no table of contents and no index — read from the top in order, or jump to a section heading that matches your question.

What is CVM++? (simple explanation)

Imagine you write instructions in a small language (`.cvm` files). CVM++ is the **program on your computer** that reads those instructions and carries them out. It does not send your code to the internet or to another app — everything happens inside one executable called `cvmp` on your machine.

CVM++ has two parts:

1. **The language** — rules for writing `.cvm` files (variables, math, `if`, functions, `print`, etc.).
2. **The toolchain** — the C++ program that understands that language: it reads, checks, translates, and runs your code.

In plain words: You are not using Python, Lua, or LLVM. The lexer, parser, compiler, and virtual machine are all implemented in this repository in C++17.

A kitchen analogy (helps many readers)

| Real life | In CVM++ | |----|----| | Recipe written in English | Your `.cvm` source file | | Reading each word | Lexer (tokens) | | Understanding recipe structure | Parser (AST tree) | | Writing numbered steps for a robot chef | Compiler (bytecode) | | Robot following numbered steps | Virtual machine (VM) | | Food on the plate | Numbers printed on the screen |

The VM is a **pretend computer** that only understands the bytecode numbers your compiler produced. Your real CPU runs the `cvmp` program; the VM runs **inside** `cvmp`.

The five steps (always in this order)

`.cvm` file → **Lexer** → **Parser** → **Compiler** → **VM** → **output**

1. **Lexer** — scans characters and groups them into tokens (words, numbers, symbols).
2. **Parser** — checks that tokens follow grammar rules and builds an AST (a tree of meaning).
3. **Compiler** — walks the tree and writes bytecode (a compact list of bytes).
4. **VM** — reads bytecode one instruction at a time and updates a stack and variables.
5. **CLI / REPL** — the terminal interface you type into (`cvmp` or `cvmp script.cvm`).

Part	What it does in one sentence
Language	Defines what valid <code>.cvm</code> programs look like.

Lexer	Turns raw text into a list of classified tokens.
Parser	Turns tokens into a structured tree (AST).
Compiler	Turns the tree into bytecode bytes.
VM	Executes bytecode and produces printed results.
CLI / REPL	Lets you run files or type code interactively.

What you need before building

Software

Tool	Why you need it
C++ compiler	Turns the project's .cpp files into the cvmpp executable. GCC 9+, Clang 10+, or MSVC 2019+.
Make	Runs the build recipe: <code>make</code> , <code>make verify</code> , <code>make clean</code> .
Git	Optional but recommended: <code>git clone</code> the repository.
CMake	Optional alternative to Make — same result, different commands.

Skills

1. Using a terminal (Terminal.app on Mac, or similar on Linux).
2. Running commands in a project folder after `cd`.
3. To **change** the C++ source, basic C++ helps but is not required to **run** examples.

Download, build, and verify (step by step)

Step 1 — Get the code

```
git clone https://github.com/Dragon9-9/cvmppl.git
cd cvmppl
```

You should see folders like `src/`, `include/`, `examples/`.

Step 2 — Compile

```
make
```

What happens: the Makefile compiles every .cpp file in `src/` with C++17 and links them into `build/cvmppl`. If you see errors, install developer tools (on Mac: `xcode-select --install`).

Step 3 — Confirm tests pass

```
make verify
```

What happens: a script runs eleven small .cvm programs and checks exit codes. You want: 11 passed, 0 failed.

Step 4 — Run one program

```
./build/cvmppl examples/hello.cvm
```

What happens: the file is read, compiled internally, executed, and you should see numbers printed (for example 42 and 50).

Why `./build/cvmpp` and not just `cvmpp`?

The executable lives in the `build/` folder. `./` means “run the program in this directory.” You can add `build` to your `PATH` later if you prefer.

Running programs (file mode)

```
./build/cvmpp path/to/script.cvm
```

The entire file is compiled and run once. Global variables start empty at the beginning of that run (unless you use the REPL — see below).

Command-line flags explained

Flag	What it does	When to use it
-d	Shows tokens, AST tree, and bytecode after compile.	Learning or debugging.
-q	Hides “Parse succeeded” banners; only program output and errors.	Scripts or demos.
-c	Compile only — does not run the VM.	Inspect bytecode with no side effects.
-h	Prints usage help.	Forgot commands.

Exit codes (what they mean)

Code	Meaning	Typical cause
0	Success	Program finished without errors.
1	Compile failure	Lexer, parser, or compiler found a mistake in your <code>.cvm</code> file.
2	Runtime failure	VM error: e.g. divide by zero, using a variable before assignment.

The interactive REPL (typing code live)

Start with no arguments:

```
./build/cvmpp
```

You get a prompt like `cvm++ >`. Type a line of CVM++ code and press Enter — you see the result immediately.

Why use the REPL?

Good for experiments: change one line, run again, see output. Variables you create with `let` stay defined for the next line (unlike file mode, which starts fresh each time).

REPL commands (with or without `:`)

You type	What happens
<code>help</code> or <code>:help</code>	Lists commands.
<code>quit</code> or <code>:quit</code>	Exits (or press Ctrl+D).
<code>debug</code> or <code>:debug</code>	Toggles debug tables on future runs.
<code>run file.cvm</code>	Runs a file; in REPL, globals can interact with session.

<code>disasm</code>	Shows bytecode from last successful compile.
<code>disasm file.cvm</code>	Compiles file without running; prints bytecode.

Multiline input

If you type `{` without a matching `}` on the same line, the prompt changes to `cvm++ {1} >`, `cvm++ {2}` `>`, ... until braces balance. Then the whole block runs as one program. This is how you paste a function body interactively.

Semicolons on one line

```
let x = 10; print x; print x + 1;
```

The REPL runs three segments in order, like three mini programs on one line.

What is inside the repository?

```
cvmpp/
├─ include/cvm++/  Header files (.hpp) – declarations
├─ src/           Source files (.cpp) – implementations
├─ examples/      Sample .cvm programs you can run
├─ docs/         Documentation (including this PDF's source)
├─ scripts/       verify.sh, build-docs.sh
├─ Makefile       Build rules
├─ CMakeLists.txt CMake build (optional)
└─ build/cvmpp    The executable (created by `make`, not in git)
```

Important source files (what each one does)

File	Role
<code>main.cpp</code>	Starts the program; REPL loop; reads CLI flags; calls compile and run.
<code>lexer.cpp</code>	Scans text into tokens.
<code>parser.cpp</code>	Builds AST from tokens.
<code>compiler.cpp</code>	AST to bytecode; functions; jumps.
<code>vm.cpp</code>	Executes bytecode on a stack.
<code>compile.cpp</code>	Connects lexer → parser → compiler.
<code>ui.cpp</code>	Colored errors and debug tables.
<code>bytecode.cpp</code>	Byte buffer; disassembler for debug.

How this project was built (learning path)

If you want to **understand** or **extend** CVM++, follow the same order the author used. Each phase adds something you can test.

Phase 1 — See tokens

Build: locations for errors, token types, lexer, simple main that prints a token table for one line.

You know it works when: `./build/cvmpp -d` shows `Let`, `Identifier`, `Integer`, etc. for `let x = 42;`

Phase 2 — See the syntax tree

Build: AST node types, recursive-descent parser, tree printer.

You know it works when: `-d` shows an indented tree for `if` and `while`.

Phase 3 — See bytecode

Build: opcode enum, byte writer, compiler for expressions and control flow.

You know it works when: `:disasm` shows `PUSH_INT`, `ADD`, `JUMP`, etc.

Phase 4 — See execution

Build: stack VM, globals, runtime errors (divide by zero).

You know it works when: `print 1 + 2;` prints 3.

Phase 5 — Polish the product

Build: REPL, file runner, user functions (`fn`, `return`, `CALL`), examples, `make verify`, GitHub CI.

The “entry JUMP” bug

Function bodies are stored at the start of the bytecode file. The VM must not start there. The compiler emits a `JUMP` at byte 0 that skips to `main`. Without this, the first instruction would be inside a function and the VM would error with `LOAD_LOCAL` outside of function.

The CVM++ language (teach-yourself section)

Below, each feature has a **short example** and **what it means**.

Variables: first time vs update

```
let count = 0;    // create variable count, value 0
count = count + 1; // update existing variable
```

`let` introduces a new name. `=` alone updates a name that already exists. Using `=` on a name you never declared with `let` is an error.

Printing

```
print 42;
print count;
```

Evaluates the expression, converts the value to text, and displays it when the run finishes (with other output lines).

If and else

```
if (x < 10) {
  print x;
} else {
  print 0;
}
```

The **condition** must be boolean (from a comparison like `<` or from `true/false`). The `else` part is optional.

While loop

```
while (n > 0) {
  n = n - 1;
}
```

Checks the condition before each iteration. There is no `break` — the condition must eventually become false, or the VM stops after a large step limit (safety).

Functions

```
fn add(a, b) {  
    return a + b;  
}  
print add(3, 4);
```

1. `fn` defines a function at the top level of the file.
2. Parameters (`a`, `b`) are local names inside the function.
3. `return` sends a value back to whoever called the function.
4. `add(3, 4)` evaluates to 7.

Recursion works: see `examples/functions.cvm` (factorial).

Input from keyboard

```
let age = input;
```

Reads one line from the keyboard, trims spaces, parses as an integer. Wrong input causes a runtime error.

Operators (what you can write)

Kind	Operators
Arithmetic	<code>+</code> <code>-</code> <code>*</code> <code>/</code> on integers
Compare	<code>==</code> <code>!=</code> <code><</code> <code>></code> <code><=</code> <code>>=</code> — result is boolean
Unary	<code>-x</code> negates an integer

What the language does not have yet

Strings (text), decimal numbers, arrays, for loops, `break/continue`, modules/files imports, or type names like `int x`. Only integers and booleans exist at runtime.

Example programs (what each file teaches)

hello.cvm

```
let x = 42;  
print x;  
if (x < 100) {  
    print x + 8;  
}
```

Lesson: `let`, `print`, `if`, comparison. Output: 42 then 50.

functions.cvm

Recursive factorial of 5 → prints 120. **Lesson:** `fn`, `return`, calling yourself.

factorial.cvm

Factorial using a `while` loop (not recursion). **Lesson:** loops and assignment.

div_by_zero.cvm

Intentionally divides by zero. **Lesson:** runtime error, exit code 2, and how the VM reports problems.

Full test list (make verify)

hello, arithmetic, booleans, if_else, factorial, multiline_demo, assignment, functions, comparisons, input_demo (stdin fed 21), div_by_zero (expects error). 11/11 must pass after a good build.

Walkthrough: reading hello.cvm line by line

```
let x = 42;
print x;
if (x < 100) {
    print x + 8;
}
```

1. **Line 1:** `let x = 42;` — create variable `x` with value 42. Internally: evaluate 42, store in global named `x`.
2. **Line 2:** `print x;` — load `x`, print it. You see 42 on screen.
3. **Line 3–5:** `if (x < 100)` is true because $42 < 100$. The block runs `print x + 8` which is 50. You see 50.

If you change `x` to 200, the condition is false and the inner print is skipped — only 200 appears once.

Alternative build with CMake

```
cmake -B build -DCMAKE_BUILD_TYPE=Release
cmake --build build
./build/cvmppl examples/hello.cvm
```

Same program, different build tool. Useful if your team standardizes on CMake.

What happens on your screen (normal run)

Without `-q`, you may see green success lines: parse succeeded, bytecode size, VM completed. Then a cyan Program output section lists what print produced. Errors appear in red with phase name and a caret — read the hint line; it often tells you exactly what to fix.

Glossary (read like a mini dictionary)

Token — One classified piece of source after lexing, e.g. the keyword `let`, the number 42, or the operator `==`. The parser never sees raw letters; only tokens.

Lexer — The module that produces tokens. Implemented in `lexer.cpp`.

Parser — The module that checks grammar and builds the AST. Uses **recursive descent**: each grammar rule is a function.

AST (Abstract Syntax Tree) — A tree of structs representing program structure. Example: a `PrintStmt` node points to an expression child. The compiler walks this tree; the VM never sees it.

Bytecode — A `std::vector<uint8_t>`: raw bytes encoding VM instructions. Faster to execute than re-reading text.

Opcode — One byte identifying an operation (`Add`, `Call`, `Halt`, ...). Extra bytes after it are **operands** (numbers, indices, jump targets).

Stack — Last-in-first-out storage for intermediate results during expressions. `1 + 2` pushes 1, pushes 2, then `ADD` pops both and pushes 3.

Global variable — A variable visible outside functions. Stored by name index in bytecode; in the REPL, also kept in a session map between lines.

Local variable — Inside a function only. Accessed by slot number (`LOAD_LOCAL` / `STORE_LOCAL`).

Call frame — When a function is called, the VM saves where to return and allocates space for parameters/locals.

Instruction pointer (IP) — Index of the next bytecode byte to execute.

Diagnostic — An error or warning with phase (LEXER, PARSER, ...), line/column, message, and often a **hint** telling you how to fix it.

VmSession — REPL-only storage so `let x = 1` on one line still gives `x` on the next line.

Front end — Lexer + parser + compiler together, before the VM runs.

When something goes wrong

Problem	What to try
cvmpp not found	Run <code>make</code> ; use <code>./build/cvmpp</code> from repo root.
Compile error in <code>.cvm</code>	Read the message and line; fix syntax; use <code>-d</code> to see stage.
Runtime error	Often divide by zero or variable used before <code>let</code> ; read VM hint.
Variables reset	File mode resets each run — use REPL to keep state.

End of Project and Build Guide · github.com/Dragon9-9/cvmpp